

Frozen Pi: Investigating Cold Boot Attacks on Embedded Systems

Ethan Duval

*The Bradley Department of Electrical and Computer Engineering
Virginia Polytechnic Institute and State University)*
Alexandria, United states of America
Ethancd19@vt.edu

Abstract—Cold boot attacks make use of a physical property of dynamic random-access memory (DRAM), called data remanence, to recover sensitive information after a full power loss. This was originally demonstrated on systems such as x86, where the dual in-line memory modules (DIMMs) can be removed from the board once cooled. This paper aims to demonstrate a streamlined approach designed for the Raspberry Pi 4 model B, a widely distributed embedded Internet of Things (IoT) device. Our approach selects a 16 MB section of the DRAM and floods this with over a million copies of a known AES-128 key, and then analyzes the extent of key recovery following controlled power loss of varying durations.

In order to simulate real-world attack conditions, the system is cooled using compressed air canisters held upside down, which causes them to expel a subzero liquid, before forcing a shutdown and reboot. This is done by manual disconnection of the GPIO GND pin to ensure that the system is awake from a “cold” state. Once this is completed the memory dumps are filtered using entropy thresholds and pattern frequency to identify key-like structures. Recovery results are benchmarked across exact matches and Hamming-distance metrics, revealing data persistence even after extended power-loss intervals. This demonstrates the continued viability of cold boot attacks on systems like modern embedded and ARM-based devices. The findings show that well-implemented data security measures are extremely important and provide a practical evaluation framework for future tests.

Index Terms—Cold boot attack, Raspberry Pi, Memory forensics, DRAM remanence, Embedded systems, key recovery

I. INTRODUCTION

Targeting data remanence, a physical property of DRAM where data is retained momentarily after power loss, cold boot attacks have the ability to recover and extract data that should no longer exist. This can be furthered by bringing the DRAM to subzero temperatures, increasing the time of decay from milliseconds

to minutes depending on the temperature. This gives adversaries more time to access sensitive data, like cryptographic keys or login credentials. The cold boot attack vulnerability was first demonstrated and publicized by Halderman et al., where they used inverted air dusters to lower DRAM temperatures to approximately -50°C and eventually submerging the DIMMs in liquid nitrogen to reach temperatures of $\sim -196^{\circ}\text{C}$. At these temperatures they saw a rate of 1% decay over 10 minutes and 0.17% decay over 60 minutes respectively [1].

With modern embedded systems like the Raspberry Pi 4 and other similar boards, these attacks still pose a threat. Unlike traditional x86 setups, the Pi does not have removable DIMMs that can be moved to a separate computer with permissions to dump the RAM contents. It instead has DRAM that is soldered to the board, meaning that we can’t use traditional techniques. Yet as shown in more recent work [3], even ARM-based devices remain vulnerable to remanence-based data recovery attacks given the right cooling and dumping mechanisms are used.

Given the rise in distribution of low-cost, memory-constrained devices like the Raspberry Pi in areas like IoT and edge computing applications, understanding how persistence of sensitive data works and can be defended against in these environments is becoming increasingly important. On top of the physical aspects of the Raspberry Pi, devices often lack features such as BIOS-level memory wiping or full disk encryption, unless they are running extremely custom software, making them an increasingly attractive target for physical attacks where the adversary has access to the device.

This paper aims to build a lightweight, reproducible setup to test cold boot attacks that does not rely on hardware modifications such as removing the soldered ram from the board. To accomplish this task, we flooded a 16 MB region of memory with around one million

copies of a known AES-128 key and then analyzed the recovery rates after varying power loss intervals. This is done either using a targeted dump that dumps the 16MB region of memory, or a full memory dump which dumps the entire 4GB of ram at boot, though takes a much longer time.

Our contributions are as follows:

- 1) We design and implement a RAM flooding mechanism for reproducible AES key injection on the Pi for testing.
- 2) We develop an entropy-based filtering and key similarity detection pipeline to quantify exact and partial key recoveries.
- 3) We demonstrate, through controlled power cycles, that significant portions of memory retain recoverable secrets even after tens of seconds offline.

The goal of this research is not to discover naturally occurring secrets, but instead to quantify data retention and decay under realistic attack conditions.

II. BACKGROUND AND RELATED WORK

A. Cold Boot Attacks and Data Remanence

Persistence of data in volatile memory has been studied for decades, but cold boot attacks gained widespread attention more recently following Halderman et al.'s 2009 work [1]. In their experiments they demonstrated that DRAM cells retain data after power is lost, and that cooling the memory significantly extends the time it takes for damaging data loss to occur. They made use of canned air and liquid nitrogen, and were able to recover cryptographic keys, disc encryption secrets, and even pictures of the Mona Lisa from x86 machines, up to 60 minutes after power was lost. These findings showed that cold boot attacks are a real-world physical threat to system confidentiality.

B. Application to Embedded and ARM-Based Devices

Recent efforts have brought cold boot studies into the world of embedded systems. Won and Bhasin [3] explored data remanence on Raspberry Pi Model B+, showing that cold boot attacks remain feasible on ARM-based distributions. Rather than using removable DIMMs like Halderman et al., they relied on using diagnostic headers and U-Boot firmware to dump RAM, enabling full memory scanning of the LPDDR2 stack. The findings of this study showed that even on devices where DRAM is soldered and traditional DIMM transfer is not an option, that it is still possible to pull sensitive data out of memory.

C. Key Detection Techniques

Tools such as aeskeyfind [2] have demonstrated that it is possible to recover AES keys in memory by identifying key schedule patterns. While these tools are effective for passive recovery of naturally occurring keys, they also assume more specific structural properties. Our system makes use of entropy filtering, pattern frequency analysis, and Hamming-distance scoring to identify repeated key sequences, including partially degraded examples, in a more generic and flexible manner.

III. METHODOLOGY

The experiment follows a sequence designed to test data remanence under realistic conditions. Each run of the experiment begins with a memory injection phase, where a custom C program floods a 16 MB region of heap memory with repeated AES-128 keys. Once the system is stabilized, the DRAM is cooled and power is abruptly disconnected using the GPIO GND pin.

Upon reboot, a dump utility runs at startup to capture a snapshot of the physical memory, before it can be altered. The captured memory is then transferred to a separate machine for post-processing.

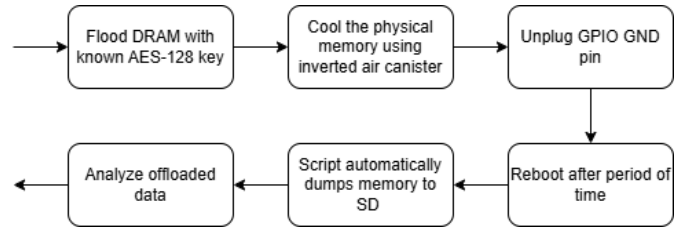


Fig. 1. a high-level overview of the cold boot attack flow used in our setup, including RAM flooding, cooling, power cycling, and memory recovery.

The dumped memory is parsed using a Python-based tool that performs the following steps:

A. Memory Analysis Pipeline

- 1) **Chunking:** The memory blob is divided into overlapping 16-byte sequences.
- 2) **Pattern Frequency:** Repeated chunks occurring more than twice are collected for further inspection.
- 3) **Entropy Filtering:** Sequences with low entropy or constant/repetitive values are filtered out to eliminate non-key-like patterns.
- 4) **Exact Match Filtering:** Recovered patterns are compared against the known AES-128 key using byte-for-byte equality.

- 5) **Fuzzy Matching:** Non-exact results are also evaluated using Hamming-distance to identify partially degraded keys.

B. Timing and Classification

To evaluate the data decay, memory dumps are captured after varying intervals without power, ranging from 0 seconds (control) up to 1 minute. A stopwatch is used to measure the time between power loss and boot, with timestamps logged automatically when the dump is complete. A key is considered recovered if the match is exact; fuzzy matches within a small Hamming-distance threshold are also logged but reported separately.

This methodology allows us to quantify how long sensitive data remains recoverable in memory after a full shutdown and how retention quality degrades with increasing delay.

IV. EXPERIMENTAL SETUP

The setup of our experiment was focused on repeatability, simplicity, and relevance to real-world embedded devices. All of the tests were conducted with the base of a Raspberry Pi 4 Model B with 4GB of LPDDR4 DRAM. The reason that this board was selected was due to its popularity in IoT and educational environments, as well as its ARM-based nature and non-removable, soldered memory layout, which fits the constraints found in most modern embedded and edge systems.

A. Memory Injection Strategy

We developed a custom C program called `hold_key.c`, that allocates a 16MB buffer on the heap and proceeds to fill it with 1,048,576 consecutive copies of our known AES-128 key. This causes a region of the RAM to be saturated with recoverable material, mimicking memory residency of sensitive data during normal operation. To ensure that all memory pages are committed, we “touch” each page by accessing every 4096th byte in the allocated buffer.

B. Power-Off and Cooling Procedure

To simulate a cold boot attack, the Pi is cooled artificially using an inverted can of compressed air. When turned upside down, the canned duster expels liquid tetrafluoroethane at subzero temperatures (approx. -50°C). The liquid is sprayed onto the DRAM for roughly 5-10 seconds. Then, power is forcibly removed by detaching the GND wire connected to the GPIO header pin 6. This process avoids the any safe shutdown routines and mimics an attack where power is abruptly cut while data is still resident in RAM, this also ensures that we

are starting from a fully turned off state, instead of a partial off state like `sleep`.

C. Automated Memory Dump on Boot

To allow for memory capture immediately after power is restored, a third C utility script named `dump.c` is executed on boot via the `/etc/rc.local` initialization script. This program reads the data from `/dev/mem` and dumps a binary image of a specified physical memory range, specified by the text file output by `virtual_to_physical.c`, to the SD card. The dumps are timestamped and can be offloaded to another machine. This was done using SCP throughout the test, though the SD card can be pulled and read on your machine similar to how the attack would be accomplished in a real-world scenario. We also developed `dump_full.c` which dumps the entire 4 GB of RAM, meaning that you don’t have to record the location of the keys. This was developed as a proof of concept, though `dump.c` was used predominantly throughout the testing due to the much shorter dump times compared to `dump_full.c`.

D. Timing and Logging

Since DRAM decay occurs rapidly, having precise control over time is critical. In our testing we use a stopwatch to manually measure the time between power loss and then the start of the memory dump, which is automatically logged in a text file alongside the dump. Future implementations of this project could automate timing using GPIO event triggers or even an external microcontroller to reduce human error.

E. Hardware and Software Summary

- **Device:** Raspberry Pi 4 Model B (4GB)
- **OS:** Raspberry Pi OS Lite (Debian-based)
- **Cooling:** Inverted Compressed air duster
- **Dumping tools:** Custom C programs (`hold_key.c`, `dump.c`)
- **Analysis:** Python scripts using entropy filtering and Hamming-distance

V. RESULTS

A. Key Recovery Over Time

Table I shows the number of exact matches and fuzzy matches (Hamming distance ≤ 1 and ≤ 2 detected after different durations of power loss. As expected, exact matches decrease with time, while partially decayed versions persist longer.

TABLE I
KEY RECOVERY VS. POWER-OFF INTERVAL

Power Loss	Exact Matches	$\leq 1\text{-Bit Off}$	$\leq 2\text{-Bits Off}$
0 sec	168756	0	0
5 sec	58368	4	30
10 sec	2033	8	25
30 sec	987	3	7
60 sec	150	0	2

While our methodology aimed to inject over 1 million key copies into RAM, the number of recoverable keys, especially in the 0-second case, was significantly lower than expected. This is likely due to fragmented physical memory pages not being captured in their entirety by out targeted or page dumping methods. Even with the technique of dumping a much larger size without dumping the entire ram, we managed to extract only 168756 exact key matches, short of the 1048576 matches that should have been recovered with the 0-second dump. This shows how memory allocation fragmentation and imperfect physical address mapping influence real-world attack viability.

B. Fuzzy Key Matches

The number of keys recovered with Hamming distances of 1 or 2 bits peaked at 8 and 30 respectively at 10 and 5 seconds. These counts quite rapidly decayed after 30 seconds. Importantly, the low frequency of fuzzy matches validates our filtering approach and indicates that only a fraction of partially corrupted keys are preserved, with most failing the entropy or structural filters.

C. Entropy of Key-like Chunks

Across all tests valid key-like chunks had entropy values that ranged from 3.5 to 4.0 bits. We observed that candidate keys matching our known AES-128 structure typically had entropy of above 3.6. Chunks with entropy below this threshold were overwhelmingly repetitive, and where either all 0s or ASCII sequences, which were successfully filtered.

D. Visualization of Decay Trends

Figure 2 shows a decay curve of exact matches over increasing power-off intervals. The steep drop between 0 and 5 seconds confirms DRAM's swift decay immediately after shutdown, with recovery rates diminishing quickly beyond 10 seconds.

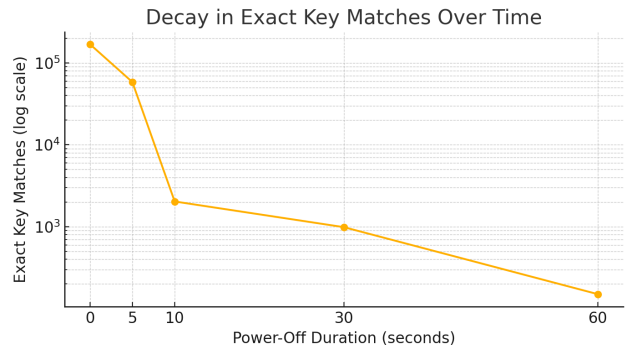


Fig. 2. Decay in exact key matches over time (log scale)

VI. DISCUSSION

Our findings validate earlier cold boot studies by showing that embedded DRAM, despite being soldered to the board and thus less accessible, is still susceptible to remanence attacks under cold conditions. However, our results also expose practical limitations. Despite injecting over a million key copies, recoverable instances were far fewer, this could be due to issues with the way that we searched, as well as potentially to do with page table translation constraints.

We also observed the decay pattern aligns with prior x86-based studies, though the absolute recovery counts are lower. Notably, our decay plateaued around 60 seconds, where only trace remnants of keys were still potentially identifiable.

Finally, the impact of dump alignment became very apparent. Early tests using physical ranges calculated from a single page mapping drastically underreported matches. Switching to 64MB dump, improved recovery but still did not guarantee the full coverage that a full dump would have.

VII. DEFENSE AND MITIGATION

While software countermeasures cannot eliminate remanence, several practical defenses can reduce the risk of cold boot attacks:

- **Boot-time Memory Wiping:** Configuring the OS or bootloader to scrub RAM at startup.
- **Use of Encrypted RAM or Secure Elements:** Employing hardware like TPM or memory encryption controllers.
- **Minimized Key Residency:** Avoid storing keys persistently in RAM; use on-demand derivation or key vaults.
- **Physical Tamper Monitoring:** Implementing hardware sensors that wipe memory or trigger shutdown upon tampering.

- **Preventing Access:** The best solution to problems like this is to prevent access to the device at all.

On systems like Raspberry Pi, many of these defenses are likely not supported by default, and would require heavily customized OS. However, awareness and use of memory hygiene techniques in application software can significantly mitigate exposure.

VIII. LIMITATIONS AND FUTURE WORK

Several technical constraints impacted the fidelity of our results:

- **Page Mapping Inaccuracy:** Virtual-to-physical address translation was estimated via the pagemap interface, which may not reflect all 16MB pages accurately.
- **Manual Dump Timing:** Timing intervals were approximated with a stopwatch. Future designs should automate shutdown and reboot timing.
- **Simplistic Injection Pattern:** Our keys were injected in bulk, not distributed throughout typical memory regions like cryptographic libraries might use.

In future work, we will ideally extend analysis to non-uniform memory distributions, experiment with AES key schedules, and attempt recovery of larger structures like RSA keys, or images and JPEG headers. Hardware refinements like microcontroller-triggered reboot cycles and integrated temperature probes will also allow much more consistent decay modeling.

IX. CONCLUSION

This research does well at demonstrating that cold boot attacks on embedded ARM devices remain quite feasible given physical access to the board and adequate cooling, the cooler the better. Even given modern hardware integration and soldered memory, keys injected into DRAM were recoverable seconds after power loss and rebooting. This duration can be increased drastically with cooler temperatures as well.

Our results further emphasize the importance of dump scope, entropy filtering, and memory hygiene. While key recoverability is something that decays quite rapidly, it persists long enough for it to pose a security risk on devices lacking mitigations. Future designs for embedded systems should adopt both hardware and software defenses to prevent remanence-based data leakage.

REFERENCES

- [1] J. Alex Halderman, Seth D. Schoen, Nadia Heninger, William Clarkson, William Paul, Joseph A. Calandrino, Ariel J. Feldman, Jacob Appelbaum, and Edward W. Felten. Lest we remember: cold-boot attacks on encryption keys. *Commun. ACM*, 52(5):91–98, May 2009.
- [2] Nadia Heninger and Ariel Feldman. aeskeyfind. <https://salsa.debian.org/pkg-security-team/aeskeyfind>, 2008. Accessed: 2025-05-04.
- [3] Yoo-Seung Won and Shivam Bhasin. Are cold boot attacks still feasible: A case study on raspberry pi with stacked memory. In *2021 Workshop on Fault Detection and Tolerance in Cryptography (FDTC)*, pages 56–60, 2021.